PanPal — Recipes, Meal Planning & Cook Mode

Master task brief · Product spec v0.9 · Platforms: iOS & Android

1. Overview

PanPal helps home cooks decide what to cook, plan the week and cook without stress. Users search recipes by what is already in their pantry, add recipes to a weekly meal plan, and get an automatically generated shopping list. While cooking, a hands-free **Cook Mode** shows one step at a time with large text, built-in timers and voice commands (“next step”, “repeat”).

2. Goals

- Find a suitable recipe in under 30 seconds using ingredient and diet filters.
- Generate a shopping list for a full week of meals in one tap, merging duplicate ingredients.
- Cook Mode that works with wet or messy hands (voice + large buttons, screen stays on).
- Launch with 2,000 curated recipes; users can also import recipes from a web link.

3. Personas

Persona	Needs
Busy parent (35)	Quick weeknight meals, kid-friendly filter, one weekly shop
Student (21)	Cheap recipes from few ingredients, cook with what is left in the fridge
Health-focused cook (42)	Nutrition per serving, vegetarian / gluten-free filters, calorie goals

4. Main features

- **Recipe search:** full-text search plus filters (ingredients I have, cooking time, diet, cuisine, difficulty).
- **Recipe detail:** photo, servings scaler (ingredient amounts update), nutrition panel, reviews.
- **Pantry:** list of ingredients at home; recipes show “you have 7 of 9 ingredients”.
- **Meal planner:** drag recipes into a 7-day × 3-meal grid; leftovers can fill a later slot.
- **Shopping list:** generated from the plan minus the pantry, grouped by supermarket aisle, shareable with family.
- **Cook Mode:** step-by-step view, parallel timers, voice commands, keep-screen-on.
- **Recipe import:** paste a URL; the backend extracts title, ingredients and steps (schema.org Recipe).

5. User flow

Photo of the flow agreed in the design sprint. The shopping list and Cook Mode are the two places where users spend most time, so they get the most polish.

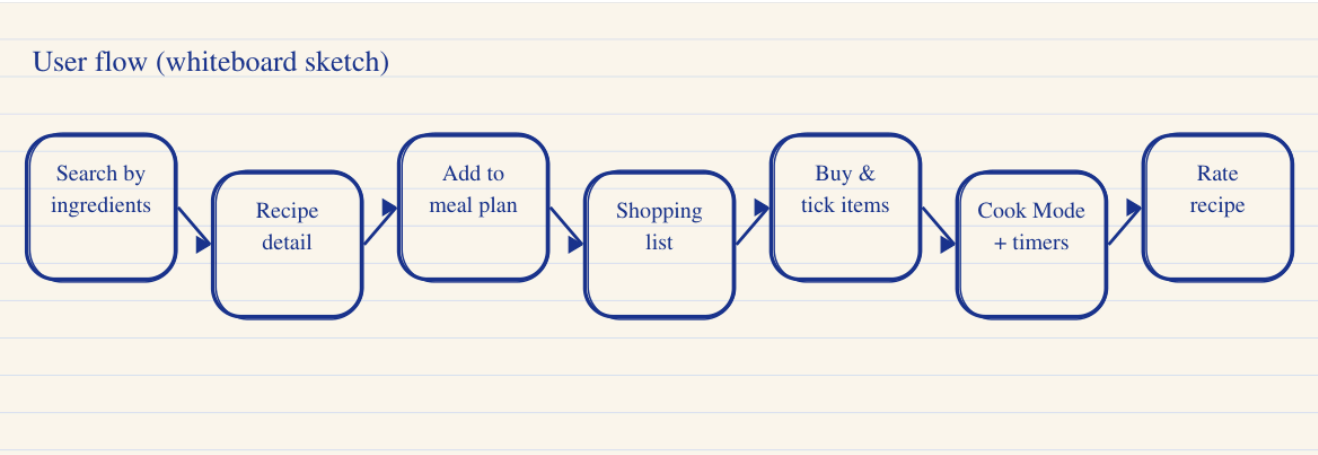


Figure 1. Core user flow (design sprint whiteboard).

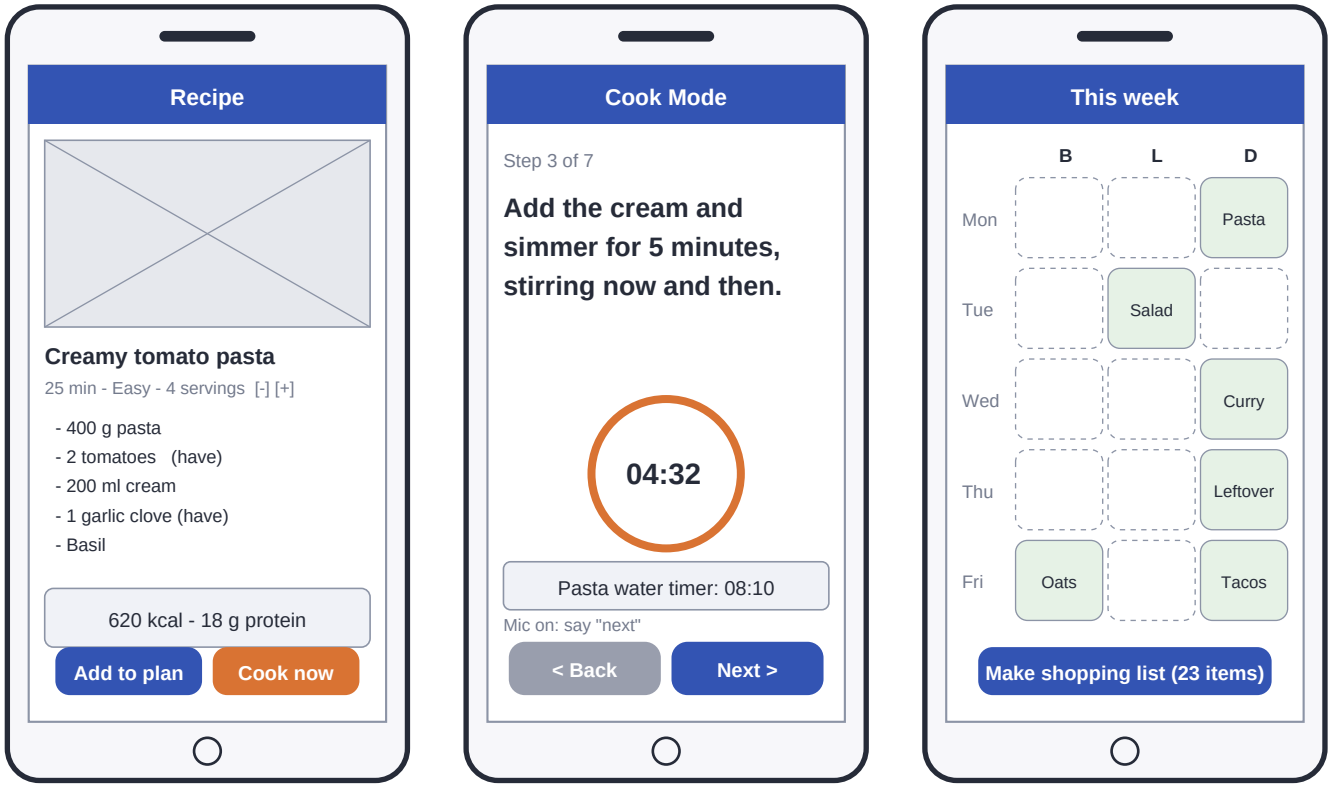


Figure 2. Wireframes: recipe detail with servings scaler, Cook Mode with timers, weekly meal planner.

6. Non-functional requirements

- Recipe search returns results in < 300 ms (p95). Recipe images are served in WebP via CDN.
- Cook Mode works offline for recipes in the current week's plan.
- Shopping list syncs between family members in real time (< 2 s).

7. Risks

- Voice commands in noisy kitchens may be unreliable — large on-screen buttons must always work.
- Recipe import depends on third-party site markup; fall back to manual editing when parsing fails.

8. Architecture

React Native app. Backend services in Node.js; search on Elasticsearch; nutrition values from an external nutrition API, cached per ingredient.

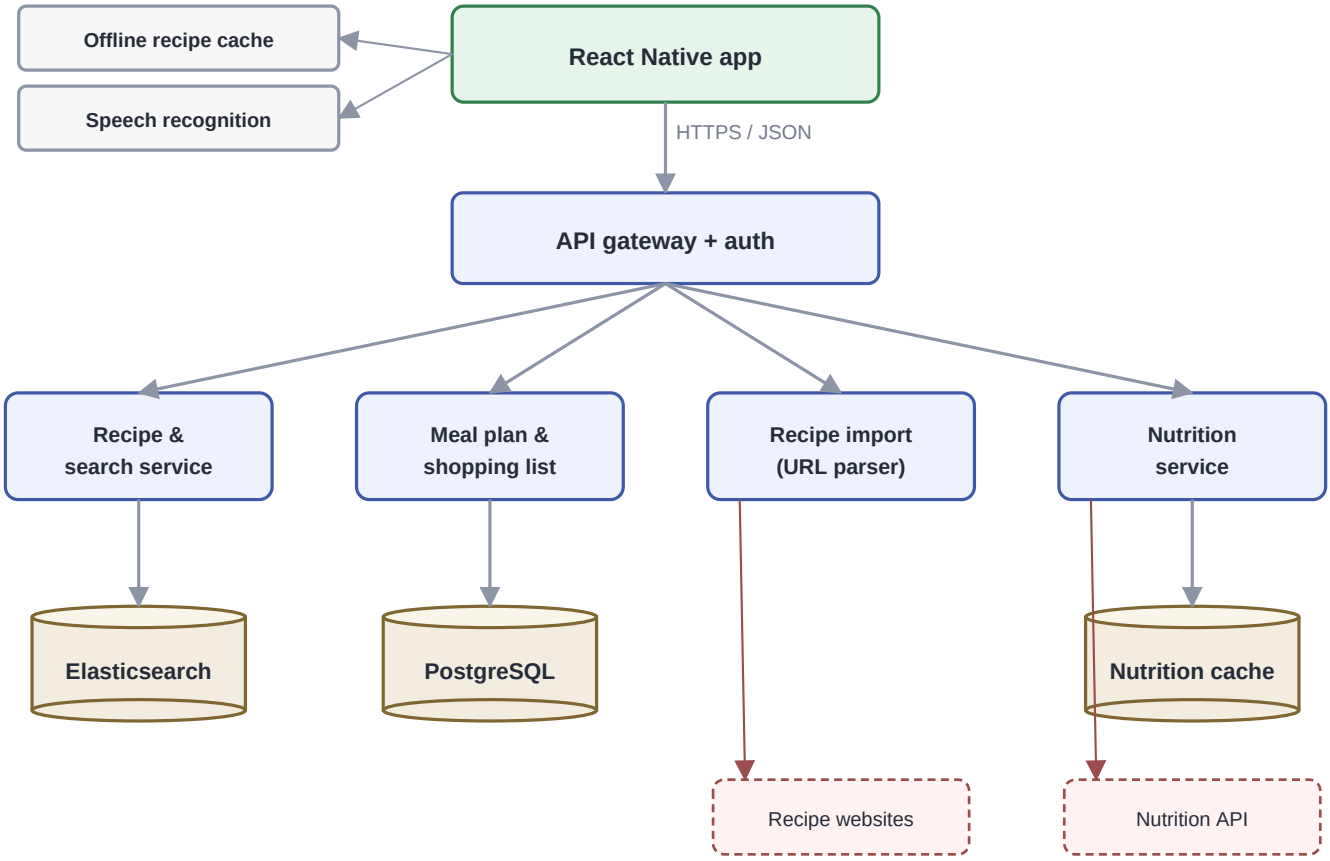


Figure 3. System architecture.

9. Data model (main entities)

Entity	Key fields
Recipe	id, title, cuisine, diet tags, time, difficulty, servings, steps[], image_url
Ingredient	id, name, aisle, nutrition per 100 g, unit conversions
MealPlan	user_id, week, slots[day][meal] → recipe_id + servings
ShoppingList	household_id, items[] (ingredient, amount, checked, added_by)

Milestones (weeks)

